Why did we design Behaviour-oriented Concurrency (BoC)?

When Concurrency Matters: Behaviour-Oriented Concurrency

LUKE CHEESEMAN, Imperial College London, UK
MATTHEW J. PARKINSON, Microsoft Azure Research, UK
SYLVAN CLEBSCH, Microsoft Azure Research, UK
MARIOS KOGIAS, Imperial College London, UK Microsoft Research, UK
SOPHIA DROSSOPOULOU, Imperial College London, UK
DAVID CHISNALL, Microsoft, UK
TOBIAS WRIGSTAD, Uppsala University, Sweden
PAUL LIÉTAR, Imperial College London, UK

The state of BoC today

6

frozen: false

balance: 100

frozen: false

balance: 50

src

dst

BoC decouples isolation and concurrency

```
transfer(50)
```

The unit of concurrency is still *Behaviours*, but they can access multiple cowns

Program state is isolated in *Cowns*

$$\text{STEP}\ \frac{E, h \hookrightarrow E', h'}{R \uplus (\overline{\kappa}, E), P, h \rightsquigarrow R \uplus (\overline{\kappa}, E'), P, h'} \qquad \text{SPAWN}\ \frac{E \hookrightarrow_{\mathsf{when}\ (\overline{\kappa'})\ \{E''\}} E'}{R \uplus (\overline{\kappa}, E), P, h \rightsquigarrow R \uplus (\overline{\kappa}, E'), P : (\overline{\kappa'}, E''), h}$$

$$\text{RUN}\ \frac{\left(\bigcup_{(\overline{\kappa'}, \_) \in (P' \cup R)} \overline{\kappa'}\right) \cap \overline{\kappa} = \varnothing}{R, P' : (\overline{\kappa}, E) : P'', h \rightsquigarrow R \uplus (\overline{\kappa}, E), P' : P'', h} \qquad \text{END}\ \frac{\mathit{finished}(E)}{R \uplus (\overline{\kappa}, E), P, h \rightsquigarrow R, P, h}$$

OOPSLA 2023

Operational Semantics